```
public class KA12 {
    public static void main(String[] args) {
        A1 o = new C1();
        System.out.print(o instanceof A1);
        System.out.print(o instanceof B1);
        System.out.print(o instanceof C1);
    }
}

A. truefalsetrue
B. falsefalsetrue
Γ. truefalsefalse
Δ. truetruetrue
```

Απάντηση/Λύση

Ο τελεστής instanceof ελέγχει το πραγματικό αντικείμενο. Στην αναφορά έχει εκχωρηθεί αντικείμενο τύπου C1. Ένα τέτοιο αντικείμενο είναι ένα C1, επίσης είναι ένα B1 και είναι ένα A1. Επομένως, ο τελεστής θα επιστρέψει true και στους τρεις ελέγχους και η σωστή απάντηση είναι η Δ.

Υπερφόρτωση συναρτήσεων

Ποια η έξοδος του κώδικα που ακολουθεί;

```
public class KA13 {

    static void m(int a) { System.out.print("int"); }
    static void m(char a) {System.out.print("char"); }

    public static void main(String[] args) {
        int i = 'a';
        char k = 127;
        m(i);
        m(k);
    }
}

A. Λάθος μεταγλώττισης
B. Λάθος εκτέλεσης
Γ. intchar
Δ. charint
```

Απάντηση/Λύση

Καταρχάς δεν υπάρχει ούτε λάθος μεταγλώττισης ούτε εκτέλεσης. Στη μεταβλητή i εκχωρείται η σταθερά ‘a’. Η σταθερά είναι τύπου χαρακτήρα. Οι χαρακτήρες όπως έχουμε εξηγήσει είναι ακέραιοι τύποι. Εκείνο που αποθηκεύεται είναι ο ακέραιος κωδικός του χαρακτήρα. Κατά την εκχώρηση i = ‘a’ λοιπόν στο i εκχωρείται μία ακέραια τιμή. Άσχετα από το ότι η τιμή αυτή προήλθε από έναν χαρακτήρα όταν προσπελαύνεται μέσω μιας ακέραιας μεταβλητής ερμηνεύεται σαν ακέραιος. Αντίστοιχα λειτουργεί και η εκχώρηση k = 127. Άσχετα με το ότι στην k εκχωρήσαμε μια ακέραιη τιμή, η k παραμένει τύπου χαρακτήρα. Επομένως, η επίλυση των κλήσεων της m στην main είναι σαφής. Στην κλήση m(i) καλείται η m με παράμετρο int και στην κλήση m(k) καλείται η m με παράμετρο char. Ο κώδικας εμφανίζει intchar και σωστή είναι η απάντηση Γ.

Σειρά αξιολόγησης

Ποια η έξοδος του κώδικα που ακολουθεί;

```
public class KA14 {
    public static void main(String[] args) {
        String s="John";
        String s1=new String(s);
        System.out.print((s==s1)+" "+s.equals(s1));
        System.out.print(" ");
        System.out.print(s==s1+" "+s.equals(s1));
    }
}

A. false true false true
B. false true true false
Γ. false false false
Δ. false true false
```

Απάντηση/Λύση

Η πρώτη print εμφανίζει false true. Η έκφραση (s==s1) είναι false, καθώς με τον τελεστή ισότητας συγκρίνονται οι αναφορές s και s1 που είναι διαφορετικές. Στη συνέχεια, η έκφραση s.equals(s1) είναι true καθώς s και s1 δείχνουν σε διαφορετικά μεν Strings που έχουν όμως και τα δύο το “John” ως περιεχόμενο.

Στη δεύτερη print όμως τα πράγματα είναι διαφορετικά. Εδώ η έκφραση s==s1 δεν περικλείεται σε παρενθέσεις, άρα δεν έχει προτεραιότητα. Ο τελεστής + έχει υψηλότερη προτεραιότητα από τον τελεστή ==. Επομένως, για να υπολογιστεί η έκφραση s==s1+” “+s.equals(s1) θα πρέπει πρώτα να υπολογιστεί η έκφραση s1+” “+s.equals(s1). Η έκφραση αυτή είναι τύπου String και η τιμή της είναι John true. Στη συνέχεια συγκρίνεται η αναφορά s με την ανώνυμη αναφορά στο String John true, οπότε η print εμφανίζει false. Συνεπώς σωστή είναι η απάντηση Γ.

Παραγωγή εξαίρεσης στο finally μπλοκ

Τι μήνυμα θα λάβουμε στη στοίβα κλήσης κατά την εκτέλεση του κώδικα που ακολουθεί;

```
class RTException extends RuntimeException {
    RTException(String msg) {
        super();
    }
}
public class KA15 {

    static void f() throws Exception {
        try {
            throw new RTException("Inside try");
        } catch (RuntimeException e) {
            throw new Exception("Inside catch");
        } finally {
            throw new RuntimeException("Inside finally");
        }
    }

    public static void main(String args[]) throws Exception {
        f();
    }
}

A. Inside try
```

```
B. Inside catch
Γ. Inside finally
```

Απάντηση/Λύση

Η παραγωγή εξαίρεσης στο finally κρύβει την κύρια εξαίρεση. Επομένως η εξαίρεση που θα φανεί κατά την εκτύπωση της στοίβας κλήσης είναι αυτή που παρήχθη μέσα στο finally, η οποία τυπώνει Inside finally. Σωστή είναι η απάντηση Γ. Το ζήτημα αυτό αντιμετωπίζεται με τη χρήση της try with resources.

Συλλογή απορριμμάτων

Έστω ο κώδικας

```
 1 package gr.ihu.cs.lmous.OOJ.kritirio2;
 2
 3 public class KA16 {
 4     private String s;
 5
 6     public KA16(String s) {
 7         this.s = s;
 8     }
 9
10     public static void f() {
11         KA16 k1=new KA16("1");
12         KA16 k2=new KA16("2");
13         k1=k2;
14         k2=null;
15         System.out.println("Hello");
16     }
17
18     public static void main(String[] args) {
19         f();
20     }
21 }
```

Επιλέξτε όλες τις σωστές προτάσεις

```
Α. Το στιγμιότυπο KA16 που δημιουργήθηκε στη γραμμή 11 είναι
διαθέσιμο προς συλλογή από τον συλλέκτη απορριμμάτων στη γραμμή
14
Β. Το στιγμιότυπο KA16 που δημιουργήθηκε στη γραμμή 12 είναι
διαθέσιμο προς συλλογή από τον συλλέκτη απορριμμάτων στη γραμμή
15
Γ. Με το πέρας της εκτέλεσης της f είναι διαθέσιμο προς συλλογή
από τον συλλέκτη απορριμμάτων μόνο το στιγμιότυπο KA16 που
δημιουργήθηκε στη γραμμή 11.
Δ. Με το πέρας της εκτέλεσης της f είναι διαθέσιμα προς συλλογή
από τον συλλέκτη απορριμμάτων και τα δύο στιγμιότυπα KA16 που
δημιουργήθηκαν στην f.
```

Απάντηση/Λύση

Το στιγμιότυπο KA16 που δημιουργήθηκε στη γραμμή 11 έχει ως μοναδική αναφορά τη μεταβλητή k1. Η μεταβλητή k1 όμως στη γραμμή 13 παύει να αναφέρεται σε αυτό το στιγμιότυπο που μένει πλέον χωρίς αναφορά. Επομένως, στη γραμμή 14 είναι διαθέσιμο προς συλλογή και άρα η Α είναι σωστή.

Στη γραμμή 14 η αναφορά k2 παύει να δείχνει στο στιγμιότυπο που δημιουργήθηκε στη γραμμή 12. Ωστόσο, στη γραμμή 13, η αναφορά k1 είχε ανακατευθυνθεί προς αυτό το στιγμιότυπο. Πράγματι η αναφορά k2 που έδειχνε αρχικά σε αυτό το στιγμιότυπο, μετά τη γραμμή 14 παύει να το δείχνει. Ωστόσο, μπορούμε να αναφερθούμε σε αυτό μέσω της αναφοράς k1. Επομένως, το στιγμιότυπο δεν είναι διαθέσιμο για να συλλεγεί ως απόρριμμα και η πρόταση Β είναι λανθασμένη.

Με το πέρας της εκτέλεσης της f, οι τοπικές μεταβλητές k1 και k2 καταστρέφονται, επομένως και τα δύο στιγμιότυπα που δημιουργήθηκαν μέσα στην f μένουν χωρίς αναφορά και άρα είναι διαθέσιμα προς συλλογή από τον συλλέκτη απορριμμάτων. Συνεπώς, η πρόταση Γ είναι λανθασμένη και η πρόταση Δ είναι σωστή.

Λίστα παραμέτρων μεταβλητού μήκους

Ποια η έξοδος του κώδικα που ακολουθεί;

```
import java.util.Arrays;

public class KA17 {

    void f(int[] a) {
        System.out.print(a[0]);
    }

    void f(int... a) {
        System.out.print(a[1]);
    }

    public static void main(String[] args) {
        KA17 k = new KA17();
        int[] t = {0, 1, 2, 3};
        k.f(t);
        k.f(0,1,2,3);
    }
}

A. 01
B. 10
Γ. Λάθος μεταγλώττισης
Δ. Λάθος χρόνου εκτέλεσης
```

Απάντηση/Λύση

Ο μεταγλωττιστής δεν μπορεί να διαφοροποιήσει μεταξύ των δύο συναρτήσεων f, καθώς είναι δυνατό και οι δύο να κληθούν με πραγματική παράμετρο ένα πίνακα ακέραιων. Σωστή απάντηση είναι η Γ.

Auto boxing

Ποια η έξοδος του κώδικα που ακολουθεί;

```
public class KA18 {
    public void f(String s) {
        System.out.print("String");
    }
    public void f(Object s) {
        System.out.print("Object");
    }

    public static void main(String[] args) {
```

```
        KA18 k=new KA18();
        k.f("Hello");
        k.f(1);
    }
}

A. StringObject
B. ObjectString
Γ. Λάθος μεταγλώττισης
Δ. Λάθος χρόνου εκτέλεσης
```

Απάντηση/Λύση

Δεν υπάρχει λάθος μεταγλώττισης, ούτε εκτέλεσης. Στην πρώτη κλήση είναι σαφές ότι ο μεταγλωττιστής επιλέγει την f με παράμετρο String, οπότε εμφανίζεται η σειρά String. Στη δεύτερη κλήση, η f καλείται με παράμετρο 1, δηλαδή με παράμετρο τύπου int. Εφόσον δεν υπάρχει f με παράμετρο int, ο μεταγλωττιστής μετατρέπει το 1 σε Integer. Η διαδικασία αυτή είναι γνωστή ως auto boxing. Είναι η ίδια λειτουργία που μας επιτρέπει να γράφουμε προτάσεις όπως:

```
Integer i=1;
```

Βεβαίως δεν υπάρχει f με παράμετρο Integer. Υπάρχει όμως f με παράμετρο Object. Καθώς ένας Integer είναι ένα Object, καλείται η f με παράμετρο Object. Επομένως, σωστή είναι η απάντηση Α.

Υπερφόρτωση και θεμελιώδεις τύποι

Ποια η έξοδος του κώδικα που ακολουθεί;

```
public class KA19 {
    public void f(int i) {
        System.out.print("int");
    }

    public void f(long l) {
        System.out.print("long");
    }

    public static void main(String[] args) {
        KA19 k=new KA19();
        k.f(1);
        int j=Integer.MAX_VALUE;
        k.f(j+1);
    }
}

A. intlong
B. intint
Γ. Λάθος μεταγλώττισης
```

Απάντηση/Λύση

Δεν υπάρχει λάθος μεταγλώττισης. Η πρώτη κλήση της f γίνεται σαφώς με ακέραια παράμετρο, οπότε ο μεταγλωττιστής καλεί την f(int) και εμφανίζει int. Κατά τη δεύτερη κλήση προστίθεται 1 στη μέγιστη ακέραια τιμή. Σε αυτήν την περίπτωση έχουμε υπερχείλιση, ωστόσο ο τύπος του αποτελέσματος είναι και πάλι int, οπότε καλείται και πάλι η f(int) και εμφανίζεται int. Επομένως, συνολικά εμφανίζεται intint και σωστή είναι η απάντηση Β.

Αλυσιδωτή κλήση

Ποια η έξοδος του κώδικα που ακολουθεί;

```
public class KA20 {

    int i=0;

    public final KA20 prt() {
        System.out.print(i++);
        return this;
    }

    public final KA20 prt(int j) {
        System.out.print(i);
        return new KA20();
    }

    public static void main(String[] args) {
        new KA20().prt().prt(1).prt(2);
    }
}
```

```
A. 1
B. 000
Γ. 010
Δ. 011
E. 012
```

Απάντηση/Λύση

Στην main δημιουργείται ένα ανώνυμο στιγμιότυπο της κλάσης KA20. Από αυτό καλείται η prt χωρίς παραμέτρους η οποία εμφανίζει το i που έχει αρχικοποιηθεί στο 0. Μετά την εμφάνισή του, το i είναι αυξημένο κατά 1 καθώς χρησιμοποιείται ο μεταθεματικός τελεστής προσαύξησης. Η κλήση στην prt() τερματίζεται με return this, δηλαδή η prt() επιστρέφει το στιγμιότυπο από το οποίο κλήθηκε και στο οποίο η μεταβλητή i εχει την τιμή 1. Με άλλα λόγια, η έκφραση new KA20().prt() έχει εμφανίσει 0, έχει μεταβάλλει την τιμή του i σε 1 και έχει επιστρέψει το στιγμιότυπο από το οποίο κλήθηκε. Από αυτό το στιγμιότυπο καλείται η prt με παράμετρο int. Αυτή καταρχάς εμφανίζει το i, δηλαδή 1 και επιστρέφει ένα νέο ανώνυμο αντικείμενο KA20. Επομένως, η έκφραση new KA20().prt().prt(1) επιστρέφει ένα νέο στιγμιότυπο. Η μεταβλητή i στο νέο στιγμιότυπο αρχικοποιείται στο 0. Από αυτό το νέο στιγμιότυπο καλείται και πάλι η prt(int) και εμφανίζει 1. Σημειώστε πως η prt(int) δεν χρησιμοποιεί την παράμετρό της. Αυτή χρησιμοποιείται μόνο από τον μεταγλωττιστή για να διαφοροποιήσει μεταξύ των δύο prt. Η σωστή απάντηση είναι η Γ.

Στατικές μέθοδοι

Επιλέξτε όσες προτάσεις είναι ορθές σε σχέση με τον κώδικα που ακολουθεί:

```
public class KA21 {

    public static void f() {
        System.out.print("f");
    }

    public void g() {
        System.out.print("g");
    }
```

```
    public static void k() {
        f();
        g();
    }

    public static void main(String[] args) {
        KA21 kA = new KA21();
        kA.k();
        KA21 kB=null;
        kB.k();
    }
}
```

```
Α. Ο κώδικας μεταγλωττίζεται όπως έχει.
Β. Υπάρχει ένα μόνο λάθος μεταγλώττισης στον κώδικα.
Γ. Υπάρχουν ακριβώς δύο λάθη μεταγλώττισης στον κώδικα.
Δ. Αν οι γραμμές που περιέχουν λάθη μεταγλώττισης διαγραφούν,
η έξοδος του κώδικα θα είναι ff.
Ε. Αν οι γραμμές που περιέχουν λάθη μεταγλώττισης διαγραφούν,
η έξοδος του κώδικα θα είναι gg.
ΣΤ. Αν οι γραμμές που περιέχουν λάθη μεταγλώττισης διαγραφούν,
ο κώδικας θα παράξει NullPointerException.
```

Απάντηση/Λύση

Η στατική μέθοδος k καλεί τη μη στατική μέθοδο g. Αυτό δεν επιτρέπεται, καθώς η k μπορεί να κληθεί και χωρίς στιγμιότυπο της KA21, ενώ η g μπορεί να κληθεί μόνο από στιγμιότυπο. Επομένως, η κλήση της g μέσα στην k παράγει λάθος μεταγλώττισης και η πρόταση Α είναι λανθασμένη. Ωστόσο, αυτό είναι το μοναδικό λάθος μεταγλώττισης στον κώδικα και η πρόταση Β είναι ορθή ενώ η Γ λανθασμένη. Αν διαγραφούν οι γραμμές που περιέχουν λάθη, δηλαδή η κλήση της g μέσα στην f, ο κώδικας θα τρέξει χωρίς να παραγάγει εξαίρεση. Στην main καλείται η k από μεταβλητή αναφοράς τύπου KA21 που έχει την τιμή null. Ωστόσο, η k είναι στατική και για την κλήση της δεν απαιτείται στιγμιότυπο παρά μόνο η κλάση στην οποία ανήκει. Καθώς ο τύπος της μεταβλητής kB είναι γνωστός, η επίλυση της κλήσης γίνεται με επιτυχία. Επομένως, η πρόταση ΣΤ είναι λανθασμένη. Εφόσον ο κώδικας θα τρέξει κανονικά με την αφαίρεση της g από την k, η έξοδός του είναι ff. Συνεπώς η πρόταση Δ είναι ορθή και η Ε λανθασμένη.

Μη στατικά μπλοκ αρχικοποίησης

Ποια η έξοδος του κώδικα που ακολουθεί;

```
class X {

    public static int i = 0;
}

public class KA22 {

    static X x1 = new X();
    static X x2 = new X();

    {
        System.out.print(x1.i);
    }

    public static void main(String[] args) {
```

```
        x1.i = 1;
        x2.i = 2;
        System.out.print(x1.i+””+x2.i);
    }
}

A.  012
B.  011
Γ.  012
Δ.  022
Δ.  12
E.  22
ΣΤ. 11
```

Απάντηση/Λύση

Η κλάση ΚΑ22 διαθέτει δύο στατικά μέλη, στιγμιότυπα της κλάσης Χ, τα x1 και x2. Κατά την εκτέλεση της main, στη μεταβλητή i της Χ εκχωρούνται διαδοχικά οι τιμές 1 και 2. Η i προσπελαύνεται μέσω των αναφορών x1 και x2, ωστόσο ως στατική μεταβλητή είναι κοινή για την κλάση Χ και ανεξάρτητη από τα στιγμιότυπα x1 και x2. Επομένως, μετά την εκχώρηση x2.i=2, η τιμή της i είναι 2 ανεξάρτητα αν την προσπελαύνουμε μέσω x1 ή x2. Το μπλοκ αρχικοποίησης πριν την main είναι μη στατικό και εκτελείται μόνο κατά τη δημιουργία στιγμιότυπων της ΚΑ22. Τέτοιο στιγμιότυπο όμως δεν δημιουργείται στον κώδικα, επομένως η print μέσα στο μπλοκ αρχικοποίησης δεν εκτελείται. Συνεπώς, η main εμφανίζει 22 και σωστή είναι η απάντηση Ε.

Μεταβλητές αναφοράς ως παράμετροι τιμής

Ποια η έξοδος του κώδικα που ακολουθεί;

```
public class KA23 {

    public static void f(StringBuilder s1, StringBuilder s2) {
        s1 = new StringBuilder("f");
        s2.append(s1);
    }

    public static void main(String[] args) {
        StringBuilder s1 = new StringBuilder("s1"),
                s2 = new StringBuilder("s2");
        f(s1, s2);
        System.out.println(s1 + " " + s2);
    }
}

A. f s2f
B. f s2
Γ. s1 s2
Δ. s1 s2f
```

Απάντηση/Λύση

Τα αντικείμενα της κλάσης StringBuilder δεν είναι αμετάβλητα όπως τα αντικείμενα της String. Η append μέσα στην f θα προσθέσει στα περιεχόμενα της μνήμης όπου δείχνει η s2 τα περιεχόμενα της μνήμης στην οποία αναφέρεται η s1. Δεδομένου ότι στην προηγούμενη γραμμή στην s1 έχει εκχωρηθεί η διεύθυνση στην οποία έχει τοποθετηθεί το f, το περιεχόμενο της s2 γίνεται s2f. Στην s1 όμως μέσα στην f εκχωρείται μια νέα διεύθυνση, δηλαδή αλλάζει η τιμή της s1 και όχι τα περιεχόμενα της μνήμης όπου αναφέρεται η s1. Καθώς η s1 είναι παράμετρος στην f, η μεταβολή αυτή ισχύει μόνο μέσα στην f, έξω από αυτήν, η s1 διατηρεί την αρχική

τιμή της. Αυτή δείχνει σε θέση όπου έχει εκχωρηθεί η τιμή s1. Επομένως στην main εμφανίζεται s1 s2f και σωστή είναι η απάντηση Δ.

Μπλοκ αρχικοποίησης

Ποια η έξοδος του κώδικα που ακολουθεί;

```
public class KA24 {

    int i = 1;
    static int j = 2;

    {
        i = 3;
    }

    static {
        j = 4;
    }

    static {
        j = 5;
    }

    {
        i = 6;
    }

    KA24() {
        i = 7;
        j = 8;
    }

    public static void main(String[] args) {
        System.out.print(KA24.j);
        System.out.print(new KA24().i);
        System.out.print(KA24.j);
    }
}

A. 878
B. 578
Γ. 678
Δ. 567
```

Απάντηση/Λύση

Η πρώτη print αναφέρεται στη στατιτκή μεταβλητή j. Σε αυτήν τη φάση έχουν τρέξει οι αρχικοποιήσεις των στατικών μεταβλητών με τη σειρά που έχουν τοποθετηθεί στον κώδικα. Επομένως, η τιμή της j είναι 5. Στη δεύτερη print δημιουργείται ένα στιγμιότυπο, οπότε τρέχουν οι μη στατικές αρχικοποιήσεις και τελικά ο δημιουργός, οπότε οι μεταβλητές i και j λαμβάνουν την τιμή 7 και 8. Η έξοδος τους κώδικα είναι 578 και σωστή είναι η απάντηση Β.

Προκαθορισμένες μέθοδοι

Ποια η έξοδος του κώδικα που ακολουθεί;

```
interface I {
    public default int f(int i) {
        return 1;
    }
}
public class KA26 implements I {

    public String f() {
        return ("2");
    }
    public String f(int i) {
        return ("3");
    }

    public static void main(String[] args) {
        KA26 k=new KA26();
        System.out.println(k.f()+""+k.f(1));
    }
}
```

```
A. 23
B. 13
Γ. Λάθος χρόνου μεταγλώττισης
Δ. Λάθος χρόνου εκτέλεσης
```

Απάντηση/Λύση

Η μέθοδος f είναι προκαθορισμένη και όχι αφηρημένη στη διεπαφή I. Μπορεί βέβαια να επανοριστεί στην KA26. Όμως στην περίπτωση που επανοριστεί θα πρέπει να έχει τιμή επιστροφής συμβατή με την f της διεπαφής. Στην προκειμένη περίπτωση όμως έχουμε δύο f(int), δηλαδή δύο f με την ίδια ταυτότητα αλλά με διαφορετική τιμή επιστροφής. Έτσι αν ο κώδικας επιτρεπόταν, η κλάση KA26 θα διέθετε μία int f(int) που κληρονομεί από τη διεπαφή και μία String f(int). Σε αυτήν την περίπτωση όμως είναι αδύνατο να επιλυθεί μία κλήση του τύπου f(int). Επομένως, έχουμε λάθος μεταγλώττισης και σωστή είναι η απάντηση Γ. Σημειώστε πως η String f() δεν δημιουργεί πρόβλημα, καθώς εδώ έχουμε υπερφόρτωση και όχι επανορισμό.

Επανορισμός μεθόδων

Επιλέξτε όσες προτάσεις είναι αληθείς.

```
abstract class KA27A {

    private void f() {
        System.out.println("KA27A");
    }
}

abstract class KA27B {

    protected void f() {
        System.out.println("KA27B");
    }
}

class KA27C extends KA27A {
```

```
    public void f() {
        System.out.println("ΚΑ27C");
    }
}

class ΚΑ27 extends ΚΑ27B {

    @Override
    public void f() {
        System.out.println("ΚΑ27");
    }
}
```

Α. Η f στην ΚΑ27C επανορίζει την f στην ΚΑ27Α
Β. Η f στην ΚΑ27C θα έπρεπε να σημειωθεί με την ετικέτα @Override
Γ. Η f στην ΚΑ27 επανορίζει την f στην ΚΑ27Β
Δ. Η f στην ΚΑ27 προκαλεί λάθος μεταγλώττισης καθώς έχει δημόσια πρόσβαση ενώ η f στην γονική έχει προστατευμένη.

Απάντηση/Λύση

Η f στην ΚΑ27Α έχει ιδιωτική πρόσβαση, δεν είναι προσβάσιμη στις παράγωγες και δεν μπορεί να επανοριστεί. Επομένως, η πρόταση Α είναι λανθασμένη. Η f στην ΚΑ27C είναι ανεξάρτητη από την f στην ΚΑ27Α και δεν χρειάζεται την ετικέτα @Override. Αντίθετα, αν την προσθέσουμε, ο μεταγλωττιστής θα παράξει λάθος. Επομένως η πρόταση Β είναι λανθασμένη. Κατά τον επανορισμό μιας μεθόδου, είναι επιτρεπτό να αλλάξει ο προσδιοριστής πρόσβασης εφόσον ο προσδιοριστής στην παράγωγη είναι ευρύτερος από τον προσδιοριστή στη γονική. Πράγματι, εδώ έχουμε προστατευμένη πρόσβαση στη γονική και δημόσια στην παράγωγη. Επομένως, οι προτάσεις Γ και Δ είναι αληθείς.

Πρόχειρο αντίτυπο υπο έκδοση εγχειριδίου από τις εκδόσεις Κάλλιπος